



대규모 시스템 설계

1. 시스템 설계

2. 핵심 요소

요구사항 분석

아키텍처 설계

데이터 모델링

스토리지

3. 주요 고려사항

확장성 (Scalability)

고가용성(HA, High Availability)

데이터베이스 분산 전략

나뉘어졌다. 트랜잭션?

일관성 - 데이터베이스 일관성 모델

통신 방식

장애 격리

5. 설계 절차

6. 그외 용어

1. 시스템 설계

- 시스템 설계(System Design): 애플리케이션이 어떻게 동작하고 구성 요소들이 어떻게 상호작용하는지 정의하는 과정.
 - 소규모에서는 잘못 설계해도 티가 안 나지만, 대규모 시스템을 잘못 설계하면 유지보수와 확장성에서 심각한 문제 발생 가능 🤖
 - 소규모, 대규모 시스템 뭐가 달라? 맛보기 표 ⬇️

구분	소규모	대규모
서버	1대	수십~수천 대 분산
DB	단일 인스턴스	샤딩, 복제, 읽기/쓰기 분리
장애	드물다	상시 일부 장애 발생 가능
일관성	트랜잭션으로 해결	CAP/PACELC 트레이드오프
배포	재시작	무중단(blue-green, canary)
디버깅	애플리케이션 로그 확인	분산 트레이싱(Distributed Tracing)
병목	보통 DB	DB → 네트워크 → 애플리케이션

- <https://ablue-1.tistory.com/109>
- <https://develop-tracking.tistory.com/entry/%EC%95%84%ED%82%A4%ED%85%8D%EC%B2%98-%EC%8B%9C%EC%8A%A4%ED%85%9C-%EC%84%A4%EA%B3%84-System-Design-%EA%B0%9C%EC%9A%94>

2. 핵심 요소

요구사항 분석

- **기능 요구사항(FR, Functional Requirements):** 무엇을 하는 시스템인가
- **비기능 요구사항(NFR, Non-Functional Requirements):** 성능(최소 지연, 처리량), 가용성, 보안, 확장성
- **규모 추정:** DAU(일일 활성 사용자 수), 피크 타임 QPS(초당 쿼리 수), 읽기/쓰기 비율, 데이터 양, 트래픽 등
 - "초당 10건"과 "초당 10만 건"은 완전히 다른 시스템!

아키텍처 설계

시스템의 전체적인 구조를 정의

스타일	특징	적합한 상황
모놀리식 아키텍처	단일 배포 단위, 단일 코드베이스	초기 단계, 소규모 팀
마이크로서비스 아키텍처(MSA, Microservice Architecture)	여러 개의 작은 서비스로 구성해 각 서비스를 독립적으로 개발, 배포, 확장. 경계를 분리 ex. 인증 서버, 기본 서비스 서버, 채팅 서버 따로 존재	조직 분리, 독립 스케일 필요
이벤트 기반 아키텍처(EDA, Event-Driven Architecture)	이벤트 발행/구독으로 결합 분리 ex. 전자상거래 시스템에서 결제 완료 이벤트 → 재고 감소/배송 시작으로 넘어감	비동기 워크플로, 다수 컨슈머
Serverless	항상 실행되지 x. 요청(이벤트)이 있을 때만 실행되고 종료됨. 인프라 관리 위임, 사용량 과금 (→ Server-based) ex. Lambda로 스케줄링 자동화	트래픽 변동 큼, 운영 비용 절감

데이터 모델링

- **RDBMS:** Tabular 정형 스키마. 강한 일관성, 트랜잭션.
- **NoSQL:** 유연한 스키마, 수평 확장 용이. 다양한 구조: 키-값(Redis), 문서(MongoDB), 컬럼(Cassandra), 그래프(Neo4j).
- **CAP 이론:** 분산 데이터베이스 시스템은 일관성(Consistency), 가용성(Availability), 분할 내구성(Partition Tolerance) 3가지 중 최대 2가지만 만족 가능
 - CA: RDBMS (일관성 ↑, 가용성 ↑)
 - CP: MongoDB, Redis (일관성 ↑, 가용성 ↓)
 - AP: Cassandra (가용성 ↑, 일관성 ↓)
 - <https://deveric.tistory.com/90>
 - <https://juniquerepo.tistory.com/74>
- **ACID (RDBMS 특징) vs BASE (NoSQL 특징)**
 - <https://jinseong-dev.tistory.com/40>

스토리지

- 이미지, 로그? 객체 스토리지 S3, GCS Bucket. / 파일? EFS ..
 - <https://www.smileshark.kr/post/amazon-s3-efs-ebs-which-aws-storage-service-to-use>

3. 주요 고려사항

| 확장성, 고가용성, 분산 데이터베이스 간 데이터 일관성, 등등

확장성 (Scalability)

시스템이 늘어나는 트래픽과 데이터를 처리할 수 있는 능력

- **스케일 업(Scale-Up)**: 기존 서버의 CPU, 메모리 스펙을 업그레이드. 서버의 컴퓨팅 성능을 높이는 것.
 - 하나의 사양을 높이는 것이므로 수직적 확장(Vertical Scaling).
- **스케일 아웃(Scale-Out)**: 비슷한 스펙의 서버를 더 추가해 부하를 분산. 컴퓨팅 수를 늘린다.
 - 비슷한 여러개가 생기는 것이므로 수평적 확장(Horizontal Scaling)
 - 단, 서버가 **stateless**여야 의미가 있다. 세션, 상태를 서버에서 관리하는데 확장해버리면 불일치가 발생한다는 뜻이다.
 - c.f. 세션 불일치 문제 해결 방법:
 - Sticky Session
 - Session Clustering
 - Session Storage
- **오토스케일링(Auto Scaling)**: 미리 설정한 기준(예: 평균 CPU 사용량 > 70%)에 따라 서버 인스턴스 수를 자동으로 늘리거나 줄임
- 인스턴스가 추가되면 **로드 밸런서(L4/L7)**가 자동으로 트래픽 분산

고가용성(HA, High Availability)

가용성 (Availability): 시스템이 중단 없이 서비스할 수 있는 능력. 서버는 항상 일부가 죽는다는 전제로 설계해야.

HA=고장이 잘 나지 않도록 하는 것

- **단일 장애 지점(SPOF, Single Point of Failure)**: 동작하지 않으면 전체 시스템이 중단되는 1개의 구성 요소.
 - ex. DB가 1개인데 DB가 먹통
 - <https://blog.seongjun.kr/single-point-of-failure/>
- **이중화/다중화(Redundancy)** : 시스템에 장애가 발생하는 걸 대비해 예비 장치를 평상시부터 중복(백업)으로 배치해 운용. 서버 이중화, DB 이중화...
 - <https://connect.kinx.net/article/81>
- **장애 조치(Failover)**: 장애 발생 시 자동으로 예비 시스템으로 전환되는 것
- **헬스 체크(Health check)**: 서버의 상태를 주기적으로 확인해 서버의 정상 작동 여부를 판단
- **로드 밸런서(L4/L7)** 핵심 역할: 서버 SPOF 제거
 - 백엔드 서버 헬스체크 → 실패하면 장애 서버를 트래픽 분산 대상에서 자동 제외, 다른 서버로 트래픽 우회
 - 로드 밸런서 자체가 SPOF 되지 않아야 → 만들 때 가용 영역을 2개 선택해 각 AZ마다 노드 생성함



가용성 잘 보장되면 너무 좋음! 그러나 Multi-AZ 옵션 등 비용 많이 발생. 얼마나 보장해야 할지 파악 중요.

데이터베이스 분산 전략

- **리플리케이션(Replication)**: 동일 데이터 복제. master-replica(slave) 구조로 읽기 부하 분산.
 - <https://theheydaze.tistory.com/593>
- **샤딩(Sharding)**: 키 기반 분할. range, hash, directory 방식. hot shard, 리밸런싱이 어려운 문제.
- **캐싱(Caching)**: Redis/Memcached로 DB 앞단 부하 흡수. cache invalidation이 가장 어려운 문제.
 - ex. 반복적 읽기 많은 경우 Look Aside, 쓰기 트래픽 너무 많은 경우 Write Back
 - <https://persi0815.tistory.com/154>
- **Read/Write 분리**: 데이터베이스에 대한 읽기 요청과 쓰기 요청을 분리하여 각각 다른 서버에 할당함으로써 부하를 분산
 - 쓰기 전용 DB(master)에 쓰고 읽기 전용 DB(slave)에 읽기
 - replication lag가 발생하는데 감수해야 <https://donghyun0826.tistory.com/113>
 - CQRS(Command Query Responsibility Segregation) 패턴
 - https://docs.aws.amazon.com/ko_kr/prescriptive-guidance/latest/modernization-data-persistence/cqrs-pattern.html

나뉘어졌다. 트랜잭션?

- **분산 트랜잭션**: MSA에서 서비스마다 DB가 분리됨 → 단일 트랜잭션 불가 → 분산 트랜잭션 필요
 - **2PC**: Prepare → Commit/Rollback 2단계. 원자성 보장.
 - 단점: 코디네이터 SPOF, 블로킹(느림), NoSQL 미지원
 - **Saga**: 긴 트랜잭션을 짧은 로컬 트랜잭션으로 쪼갬. 실패하면 이전까지의 작업을 보상(Compensation) 트랜잭션으로 되돌림. 비동기.
 - Choreography: 서비스끼리 이벤트 pub/sub (Kafka 등)
 - Orchestration: 중앙 오케스트레이터가 흐름 관리
 - <https://velog.io/@ch200203/MSA-%ED%99%98%EA%B2%BD%EC%97%90%EC%84%9C%EC%9D%98-%EB%B6%84%EC%82%B0-%ED%8A%B8%EB%9E%9C%EC%9E%AD%EC%85%98-%EA%B4%80%EB%A6%AC2PC-SAGA-%ED%8C%A8%ED%84%B4>
 - <https://www.msap.ai/docs/msa-expert-from-concepts-to-practice/part-1-msa-fundamentals/chapter-2-core-concepts-of-msa/section-2-5-data-management/subsection-2-5-3-distributed-transactions/>
- **이벤트 소싱(Event Sourcing)**: 상태가 아닌 이벤트를 저장
 - 현재 잔액 500원 저장 X → [입금 1000, 출금 300, 출금 200] 이벤트 저장 O
 - 변경 이력 완전 보존. Saga와 조합해 쓰임 → 실패 지점부터 재처리 가능
 - <https://azderica.github.io/01-architecture-msa/>

- https://docs.aws.amazon.com/ko_kr/prescriptive-guidance/latest/cloud-design-patterns/event-sourcing.html

일관성 - 데이터베이스 일관성 모델

데이터베이스 여러 대 → 볼 때마다 얼마나 일관적이어야?

Strong Consistency, Eventual Consistency

▼ 모델

- **Strong Consistency (강한 일관성):** 쓰기가 성공한 시점 이후의 모든 읽기는, 어느 노드에 요청하든 그 쓰기 또는 그 이후의 쓰기 결과를 본다.
 - 모든 노드가 동일한 값을 보여주는 것이 보장됨. 대신 동기화를 기다려야 하므로 지연이 커지고 가용성이 떨어짐.
- **Eventual Consistency (최종 일관성):** 쓰기가 즉시 모든 노드에 반영되지 않음. 시간이 지나면 모든 복제본이 같은 상태로 수렴함.
 - 갱신 직후에는 노드마다 다른 값을 볼 수 있음 (구버전이 보일 수 있음)
 - 동시 쓰기가 충돌할 수 있어 해결 방법이 필요함. 예: LWW(Last-Write-Wins) — 타임스탬프가 더 최신인 쓰기를 채택하고 나머지는 버림
 - NoSQL: 분산 노드. Scale-Out 및 샤딩해서 씀 → 대개 최종 일관성을 보장
 - <https://www.geeksforgeeks.org/system-design/does-mongodb-use-eventual-consistency/>
- Strong Eventual Consistency (강한 최종 일관성): 수렴 보장에 더해 "충돌이 결정적으로 자동 해결된다"가 추가된 모델. 같은 업데이트 집합을 받은 노드는 받은 순서와 무관하게 항상 동일한 최종 상태에 도달함.
 - ▼ CRDT 동작 원리
 - 핵심은 자료구조의 병합 함수 `merge(A, B)` 를 교환·결합·역등성이 성립하도록 설계하는 것. 그러면 노드들이 업데이트를 어떤 순서로 받든, 같은 메시지를 중복으로 받든, 결국 같은 상태에 도달함.
 - 충돌 시 한쪽을 버리지 않고 그 자료구조의 규칙대로 합침
예: 카운터는 숫자 올리기, Set이면 합치기, 단일 슬롯이면 마지막걸로 하나 고르기 등
- 참고로 일관성 ≠ 멱등성(Idempotency), 무결성(integrity).

통신 방식

동기 vs 비동기

- **Synchronous (동기):** 호출자가 응답을 받을 때까지 대기. 흐름이 직관적이지만 장애 전파.
- **Asynchronous (비동기):** 호출자가 응답을 기다리지 않고 다음 작업 진행. 결합도 낮지만 추적·디버깅 비용 증가.
 - <https://mezzaninex.tistory.com/5643>

통신 방향

- **Request-Response:** 호출자가 수신자 지정 (1:1)

- **Push:** 서버가 클라이언트로 즉시 전송 (1:1, 연결 기반)
- **Event-Driven / Pub-Sub:** 발행자가 수신자를 모른 채 이벤트 발행 (1:N)

그외

▼ 프로토콜별 통신 스타일

스타일	프로토콜	특징
동기 Req-Res	REST / HTTP	리소스 기반 범용
동기 Req-Res	gRPC (unary)	스키마 기반 RPC
동기 Req-Res	GraphQL	단일 엔드포인트 쿼리 기반
비동기 Req-Res	gRPC (streaming)	커넥션 기반 스트림
비동기 Push	WebSocket	양방향
비동기 Push	SSE	단방향 (서버 → 클라)
비동기 Event-Driven	Webhook	서버 → 서버 HTTP 콜백
비동기 Event-Driven	AMQP, MQTT, Kafka	브로커 경유 Pub-Sub

▼ 데이터 통신 패턴

패턴	호출자 기준	동작	기술 예시
Request-Response	동기	요청 보내고 응답 대기	REST, gRPC
Pub-Sub	비동기	브로커 경유, 발행자/구독자 분리	Kafka, MQTT
Short Polling	동기 (반복)	클라이언트가 주기적으로 요청 (pull)	HTTP
Long Polling	동기 (지연 응답)	연결 유지 후 업데이트 시 응답	HTTP
Push	비동기	서버 → 클라이언트 즉시 전송	WebSocket, SSE
Webhook	비동기	미리 callback URL 등록, 이벤트 발생 시 서버 → 서버 HTTP 호출 (HTTP 자체는 동기)	HTTP

▼ 메시지 브로커 모델

비동기 Event-Driven을 구현하는 두 가지 브로커 방식

- Message Queue (RabbitMQ, SQS): 소비 시 메시지 삭제. 워커 풀 작업 분배용.
- Event Stream (Kafka): 메시지 보존, 오프셋으로 위치 관리. 재처리·다중 독립 소비 가능.

메시지 전달 보장 수준

- **At-most-once:** 최대 1번 전달. 유실 허용, 중복 없음. 로그·메트릭용.
- **At-least-once:** 최소 1번 전달. 디폴트. 유실 없음, 중복 가능 → 컨슈머 멱등 처리 필요.
- **Exactly-once:** 정확히 1번 전달. 스루풋·복잡도 비용 큼.

Fan-out(팬아웃): 이벤트를 여러 Consumer Group에 동시에 전달하는 패턴

장애 격리

- **연쇄 장애(Cascading Failure):** 한 컴포넌트의 장애가 다른 컴포넌트의 장애를 유발하여 전체 시스템이 마비되는 것. → 대응 전략 있어야 시스템의 복원력(Resilience) 높임

어떻게?

- **서킷 브레이커(Circuit breaker):** 다운스트림 장애 시 호출 차단
 - 누전 차단기. 두꺼비집. 외부 서버가 장애났을 때, 복구전까지 한동안 시도를 아예 안해야 하니까.
 - <https://mangkyu.tistory.com/261>
- **타임아웃(Timeout):** 무한 대기 방지
 - **Connection Timeout:** 클라이언트에서 설정한 시간까지 서버에 연결되지 않으면 발생
 - **Read Timeout:** 클라이언트에서 설정한 시간까지 서버에 응답이 오지 않으면 발생
 - <https://docs.tosspayments.com/resources/glossary/timeout#read-timeout>
- **Retry + exponential backoff + jitter:** 재시도를 점점 더 띄엄띄엄, 지수함수 형태로 sleep한뒤에 하기.
 - <https://velog.io/@dasd412/Retry-with-Exponential-Backoff-%EA%B5%AC%ED%98%84%ED%95%98%EA%B8%B0-with-jitter>
- **Bulkhead:** 리소스 풀 분리로 한 영역 장애가 전체 자원을 잡아먹지 않게
 - <https://jinwookoh.tistory.com/243>

5. 설계 절차

1. **요구사항 정리:** 기능, 비기능, 제약
2. **용량 산정:** QPS, 저장 용량, 대역폭, 메모리
3. **API 설계:** REST/gRPC/GraphQL, 인증, 버저닝
4. **데이터 모델 설계:** 스키마, 인덱스, 파티셔닝 전략
5. **고수준 아키텍처:** 컴포넌트 다이어그램, 데이터 흐름
6. **상세 설계:** 병목 예상 지점 깊게 (캐시 정책, 샤딩 키, 큐 구조)
7. **장애 시나리오 점검:** 어디가 죽으면 뭐가 멈추는가, 어떻게 복구하는가
8. **트레이드오프 정리:** 왜 이 선택을 했는지 명시

6. 그외 용어

성능 지표

- Latency (지연 시간): 요청에서 응답까지.
 - p50, p95, p99
- Throughput (처리량): 단위 시간당 처리량.
 - **TPS** (Transactions Per Second), **RPS** (Requests Per Second)
- Bottleneck (병목 지점): 가장 느린 구간

옵저빌리티 (Observability, 관측성)

- Logs: 무엇이 일어났는가
- Metrics: 얼마나 자주, 얼마나
- Traces: 요청이 어디를 거쳐 어디서 느려졌는가

기타

- 데이터 레이크. 데이터 웨어하우스. 스토리지.

- 프록시
- 게이트웨이

[AWS] 📖 클라우드 용어 정리 (고가용성 / 장애내구성 / 확장성 / 탄력성)

AWS 클라우드 용어 정리 AWS 관련된 자료를 보다 보면 많은 단축용어들이 준비하게 있다. 그래서 용어 뜻을 몰라 설명을 봤더니 알수없는 전문용어들이 나와 또 찾아보게되고 꼬리를 물고 물리는 상황이 연출된다. 예를들어, 고가용성 이니 장애허용 이니 각 구성된 단어의 뜻은 알

<https://inpa.tistory.com/entry/AWS-%F0%9F%93%9A-%EC%9E%90%EC%A3%BC-%EC%93%B0%EC%9D%B4%EB%8A%94-aws-%EB%84%A4%ED%8A%B8%EC%9B%8C%ED%82%B9-%ED%81%B4%EB%9D%BC%EC%9A%B0%EB%93%9C-%EC%9A%A9%EC%96%B4-%EC%A0%95%EB%A6%AC>

과제 분석 시 점검할 축

- 고가용성, 확장성
- 데이터 흐름 (어디서 생성 → 어디로 저장 → 어디서 소비)
- 데이터 일관성 요구 수준
- 트래픽 패턴 (균일(낮에만?)/스파이크/계절성)
- 장애 시나리오 (무엇이 죽었을 때 무엇이 멈추는가)



모든 설계는 트레이드오프다. 정답은 없고, 요구사항과 제약 안에서 무엇을 포기하고 무엇을 지킬지 결정하는 것이 설계.

